B28: SQL SuperHero Program : Session 16

1 message

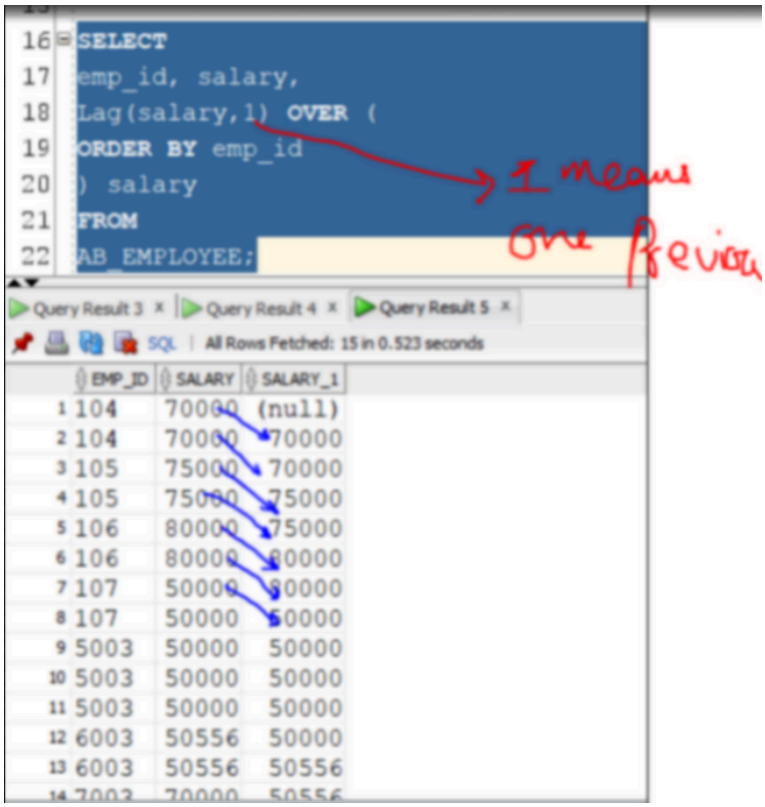
Vishal Jaiswal <vishaldbdev@gmail.com> 11 March 2025 at 01:06
Bcc: satishhattekar1997@gmail.com

```
CREATE TABLE AB_EMPLOYEE
  ( EMP_ID VARCHAR2(5 ),
EMP_NAME VARCHAR2(20 ),
DEPT_ID VARCHAR2(5 ),
EXPERTISE VARCHAR2(50 ),
SALARY NUMBER(10,2),
RESULTS VARCHAR2(10 )
  );

Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('5003','ABINASH','1','SCIENCE','PASS',50000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('5003','ABINASH','1','ENGLISH','PASS',50000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('5003','ABINASH','1','MATH','PASS',50000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('107','AMARESH','2','MATH','PASS',50000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('107','AMARESH','2','ENGLISH','PASS',50000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('105','JYOTI','3','MATH','FAIL',75000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('105','JYOTI','3','ENGLISH','PASS',75000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('7003','NISHAD','2','ENGLISH','FAIL',70000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('7003','NISHAD','2','MATH','PASS',70000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('6003','RAKESH','2','MATH','PASS',50556);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('6003','RAKESH','2','ENGLISH','FAIL',50556);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('104','RAVI','2','MATH','PASS',70000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('104','RAVI','2','ENGLISH','PASS',70000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('106','REDDY','2','MATH','FAIL',80000);
Insert into AB_EMPLOYEE (EMP_ID,EMP_NAME,DEPT_ID,EXPERTISE,RESULTS,SALARY) values ('106','REDDY','2','ENGLISH','PASS',80000);
```

The LAG() function can be very useful for comparing the value of the current row with the value of the previous row.
The LEAD() function can be very useful for comparing the value of the current row with the value of the following row.

Run queries on the same table we created above



```
SELECT
emp_id, salary,
Lead(salary,1) OVER (
ORDER BY emp_id
) salary
FROM
AB_EMPLOYEE;
```

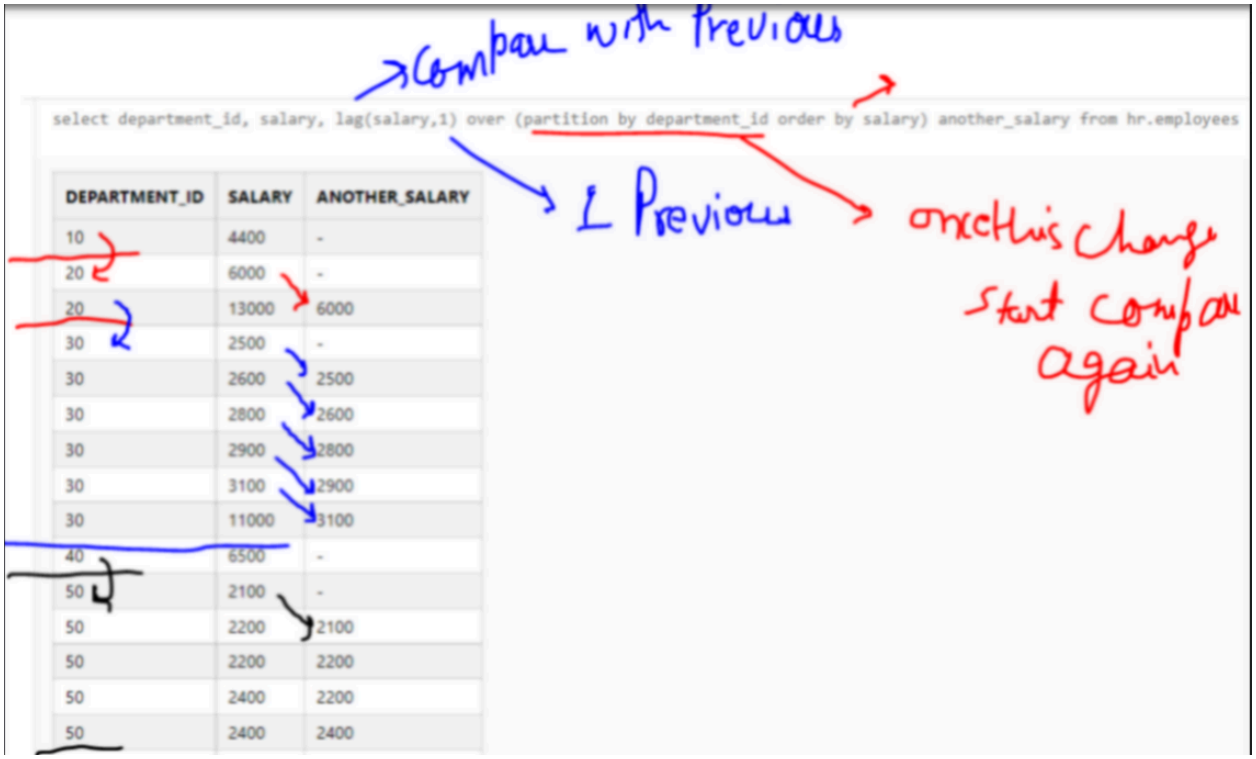
Query Result 3 | Query Result 4 | Query Result 5

All Rows Fetched: 15 in 0.523 seconds

	EMP_ID	SALARY	SALARY_1
1	104	70000	70000
2	104	70000	75000
3	105	75000	75000
4	105	75000	80000
5	106	80000	80000
6	106	80000	50000
7	107	50000	50000
8	107	50000	50000
9	5003	50000	50000
10	5003	50000	50000
11	5003	50000	50556
12	6003	50556	50556
13	6003	50556	70000

==========

select department_id, salary, lag(salary,1) over (partition by department_id order by salary) another_salary from hr.employees;



DEPARTMENT_ID	SALARY	ANOTHER_SALARY
10	4400	-
20	6000	-
20	13000	6000
30	2500	-
30	2600	2500
30	2800	2600
30	2900	2800
30	3100	2900
30	11000	3100
40	6500	-
50	2100	-
50	2200	2100
50	2200	2200
50	2400	2200
50	2400	2400

See this above query. I am comparing "Partition by department id" means once department id changes we start comparing again. For the first records department_id = 10, there is only one employee, so salary 4400 is coming but no other employee in the same department to compare so the next column is blank. Next records, department id = 20, starts again. what sharts again??? Compare starts again. for the first record with 6000 salary, no one before that to compare so next record is null, but another employee in the same department 20, we have salary 13000 and previous have salary 6000.

==========

Input:

Weather table:

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

Output:

id
2
4

Explanation:

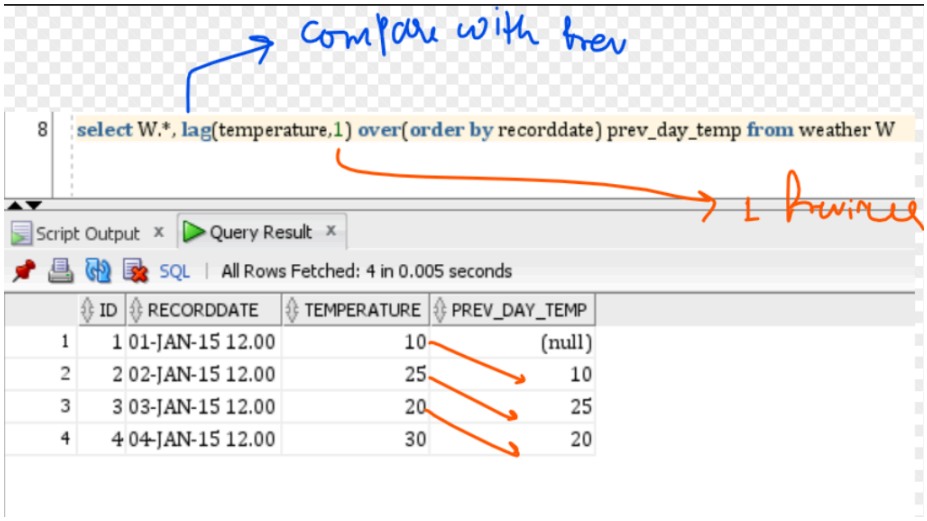
In 2015-01-02, the temperature was higher than the previous day (10 -> 25).
In 2015-01-04, the temperature was higher than the previous day (20 -> 30).

See the above scenario, we discussed in class. **Write a sql to give me the date when the temperature is higher than the previous day**.

Solution: Understand the question here. we want to find out the date, when temp is higher than previous. like in this example row2 and row4 is answer, why and how ? Why because on row2, temp is 25 and previous is 10 and same with row4 the temp is 40 and prev is 20. We have to compare the temp and compare means lead and lag. here compared with previous day means lag.

```
create table weather(id number, recorddate date, temperature number);

insert into weather values (1, '01-JAN-2015',10);
insert into weather values (2, '02-JAN-2015',25);
insert into weather values (3, '03-JAN-2015',20);
insert into weather values (4, '04-JAN-2015',30);
```



We used lag function and get the previous day's temperature. Now the requirement is when the temp is more than previous day. now just compare these 2 columns.
\

```
8  with N1 as (
9  select W.*, lag(temperature,1) over(order by recorddate) prev_day_temp from weather W)
10 select * from n1 where temperature > prev_day_temp
```

Script Output | Query Result

SQL | All Rows Fetched: 2 in 0.005 seconds

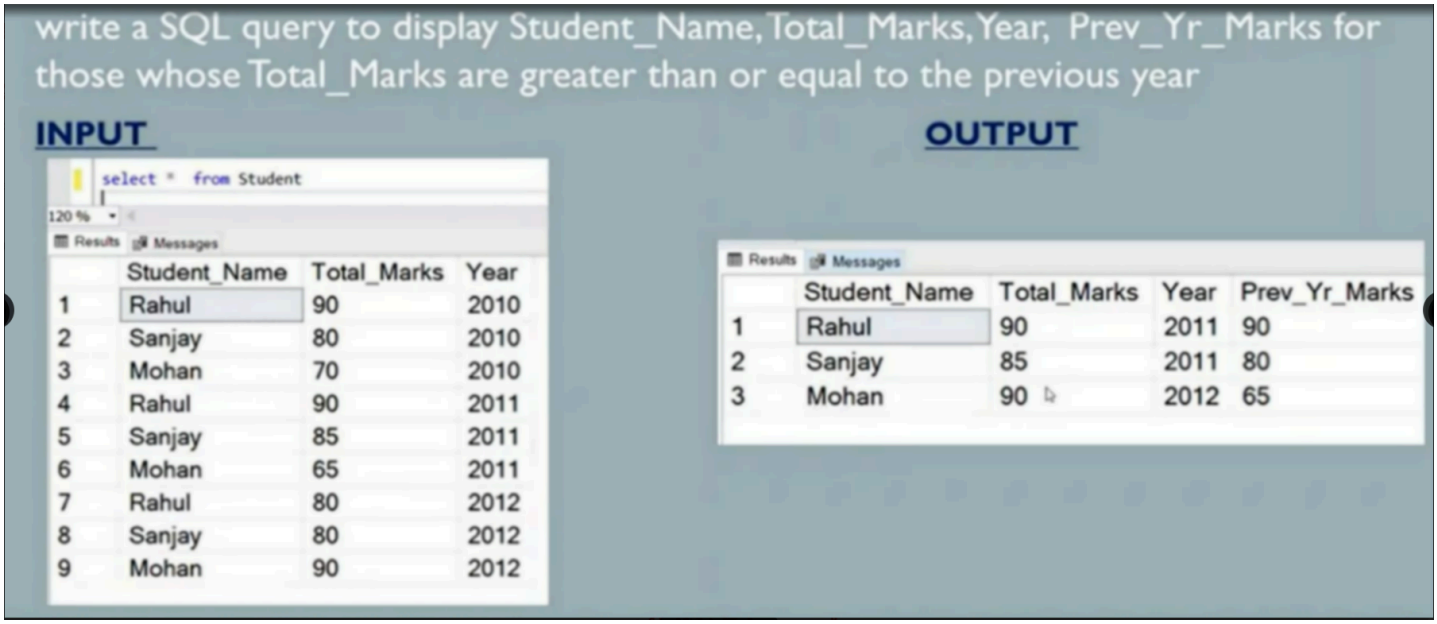
	ID	RECORDDATE	TEMPERATURE	PREV_DAY_TEMP
1	2	02-JAN-15 12.00	25	10
2	4	04-JAN-15 12.00	30	20

got the o/p

==========

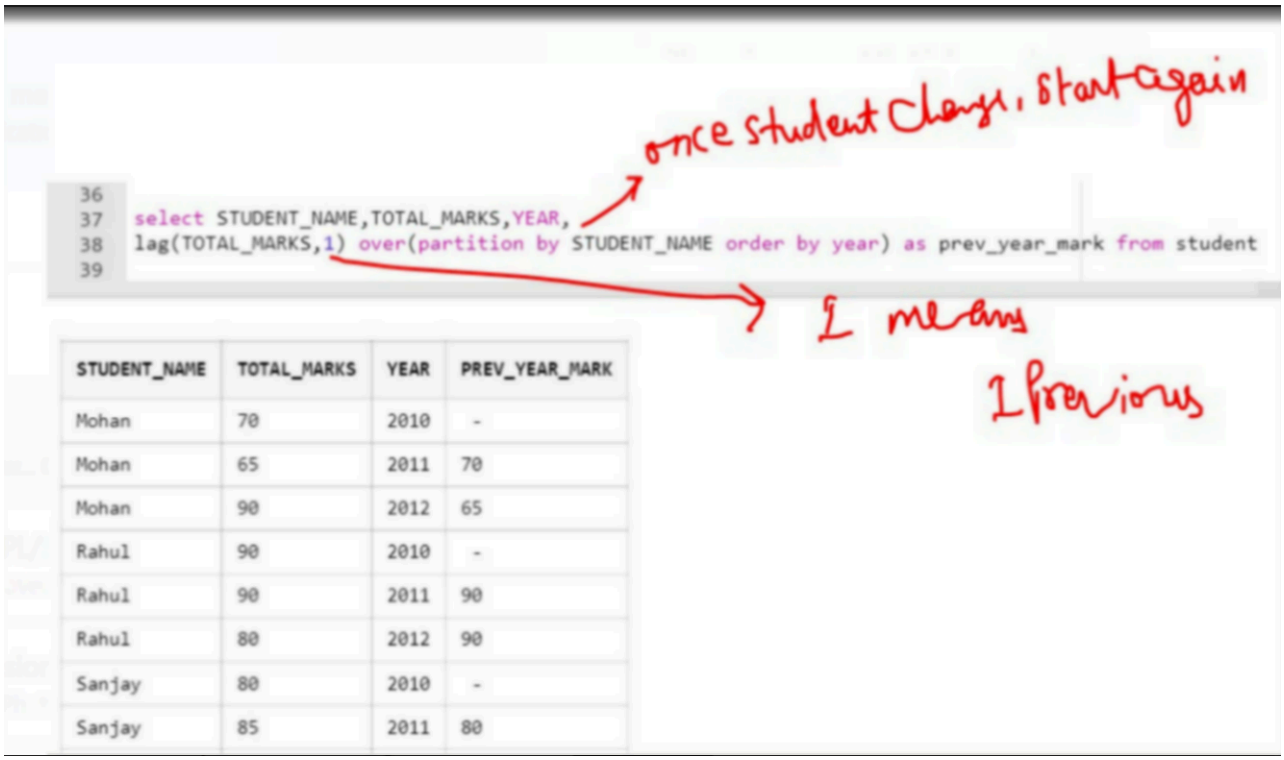
```
create table student(
student_name varchar2(20),
totaL_marks number,
year number
);

insert into student values ('Rahul',90,2010);
insert into student values ('Sanjay',80,2010);
insert into student values ('Mohan',70,2010);
insert into student values ('Rahul',90,2011);
insert into student values ('Sanjay',85,2011);
insert into student values ('Mohan',65,2011);
insert into student values ('Rahul',80,2012);
insert into student values ('Sanjay',80,2012);
insert into student values ('Mohan',90,2012);
```



question is to compare previous year marks. it means, whenever we have see this word : compare with previous or next that is leag and lead.
before start writing query , lets focus the trick.whenever we have to talk about previous means --> analytical function --> lag

select STUDENT_NAME,TOTAL_MARKS,YEAR, lag(TOTAL_MARKS,1) over(partition by STUDENT_NAME order by year) as prev_year_mark from student;




```
36
37  select STUDENT_NAME,TOTAL_MARKS,YEAR,
38  lag(TOTAL_MARKS,1) over(partition by STUDENT_NAME order by year) as prev_year_mark from student
39
```

STUDENT_NAME	TOTAL_MARKS	YEAR	PREV_YEAR_MARK
Mohan	70	2010	-
Mohan	65	2011	70
Mohan	90	2012	65
Rahul	90	2010	-
Rahul	90	2011	90
Rahul	80	2012	90
Sanjay	80	2010	-
Sanjay	85	2011	80


```
37  select STUDENT_NAME,TOTAL_MARKS,YEAR,
38  lag(TOTAL_MARKS,1) over(partition by STUDENT_NAME order by year) as prev_year_mark from student
39
```

STUDENT_NAME	TOTAL_MARKS	YEAR	PREV_YEAR_MARK
Mohan	70	2010	-
Mohan	65	2011	70
Mohan	90	2012	65
Rahul	90	2010	-
Rahul	90	2011	90
Rahul	80	2012	90
Sanjay	80	2010	-
Sanjay	85	2011	80
Sanjay	80	2012	85

partition by creates the boundary here.because for each student, we have to compare previous year marks. sp each means ----------> group by. we are doing comparison, so we are using --> analytical function ----------> Lag. because of analytical function, group by is chnaged to ----------> partition by

with n1 as (
select STUDENT_NAME,TOTAL_MARKS,YEAR, lag(TOTAL_MARKS,1) over(partition by STUDENT_NAME order by year) as prev_year_mark from student
) select * from N1 where TOTAL_MARKS >= PREV_YEAR_MARK;

```
with n1 as (
select STUDENT_NAME, TOTAL_MARKS,YEAR, lag(TOTAL_MARKS,1) over (partition by STUDENT_NAME order by year) prev_year_mark
from student
) select * from n1 where TOTAL_MARKS >= PREV_YEAR_MARK
```

STUDENT_NAME	TOTAL_MARKS	YEAR	PREV_YEAR_MARK
Mohan	90	2012	65
Rahul	90	2011	90
Sanjay	85	2011	80

3 rows selected.

3rd Parameter is default value

```
SELECT *,
       Lag(JoiningDate, 1,'1999-09-01') OVER(
       ORDER BY JoiningDate ASC) AS EndDate
FROM @Employee;
```

In the output, we can see a default value instead of NULL in the first row:

	EmpCode	EmpName	JoiningDate	EndDate
1	3	Sonu	2018-03-10	1999-09-01
2	1	Rajendra	2018-09-01	2018-03-10
3	2	Manoj	2018-10-01	2018-09-01
4	4	Kashish	2018-10-25	2018-10-01
5	6	Akshita	2018-11-01	2018-10-25
6	5	Tim	2018-12-01	2018-11-01

Default value

null replace by default

creates boundary

```
SELECT [Year],
       [Quarter],
       Sales,
       LAG(Sales, 1, 0) OVER(PARTITION BY [Year]
       ORDER BY [Year],
                [Quarter] ASC) AS [NextQuarterSales]
FROM dbo.ProductSales;
```

Previous

In the following screenshot, we can see three partitions of data for 2017,2018 and 2019 year. The Lag function individually works on each partition and calculates the required data:

	Year	Quarter	Sales	NextQuarterSales
1	2017	1	55000.00	0.00
2	2017	2	78000.00	55000.00
3	2017	3	49000.00	78000.00
4	2017	4	32000.00	49000.00
5	2018	1	41000.00	0.00
6	2018	2	8965.00	41000.00
7	2018	3	69874.00	8965.00
8	2018	4	32562.00	69874.00
9	2019	1	87456.00	0.00
10	2019	2	75000.00	87456.00
11	2019	3	96500.00	75000.00
12	2019	4	85236.00	96500.00

1 2 3

==========

employee_ID	salary	year
1	80000	2020
1	70000	2019
1	60000	2018
2	65000	2020
2	65000	2019
2	60000	2018
3	65000	2019
3	60000	2018

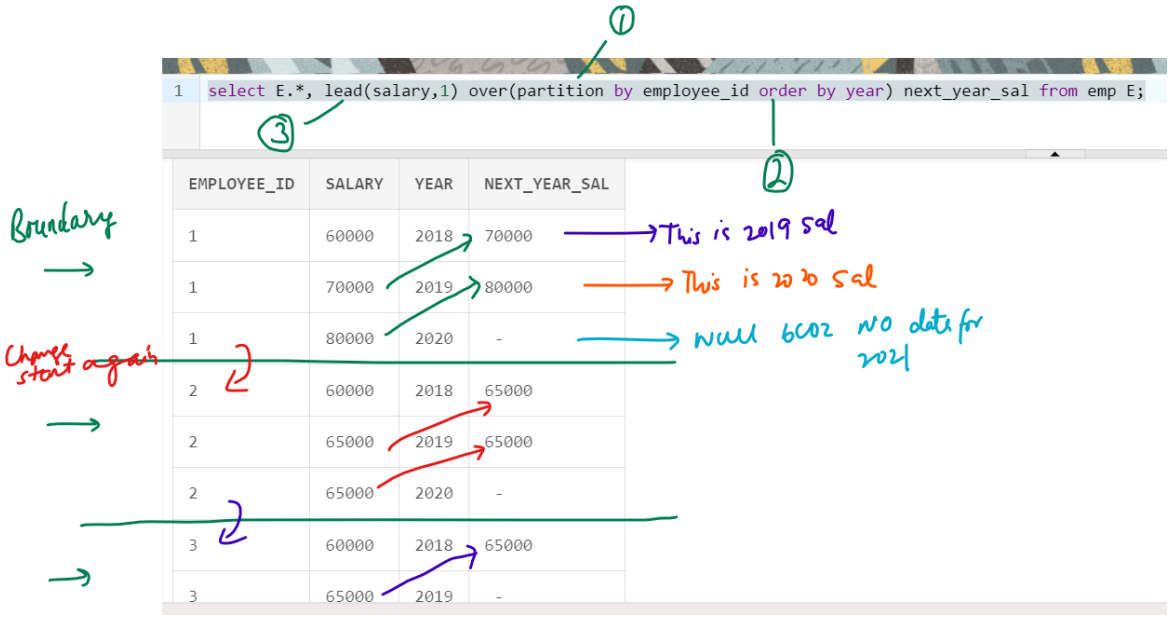
If you see, Employee 1 is the only employee that has a three year increase, so how can we do that. if we know the samary of 3 years and able to compare. We know the salary of 2018, if want to see 2019(next year sal) and 2020 (next to next year sal).

lead(salary,1) over(partition by employee_id order by year) next_year_sal

lead --> next year
salary --> this is what i want to compare
1 --> 1 next

partition by employee_id order by year

partition by employee_id--> for each employee, i want to run lead for.
order by year --> arrange the data, after partition and then run lead

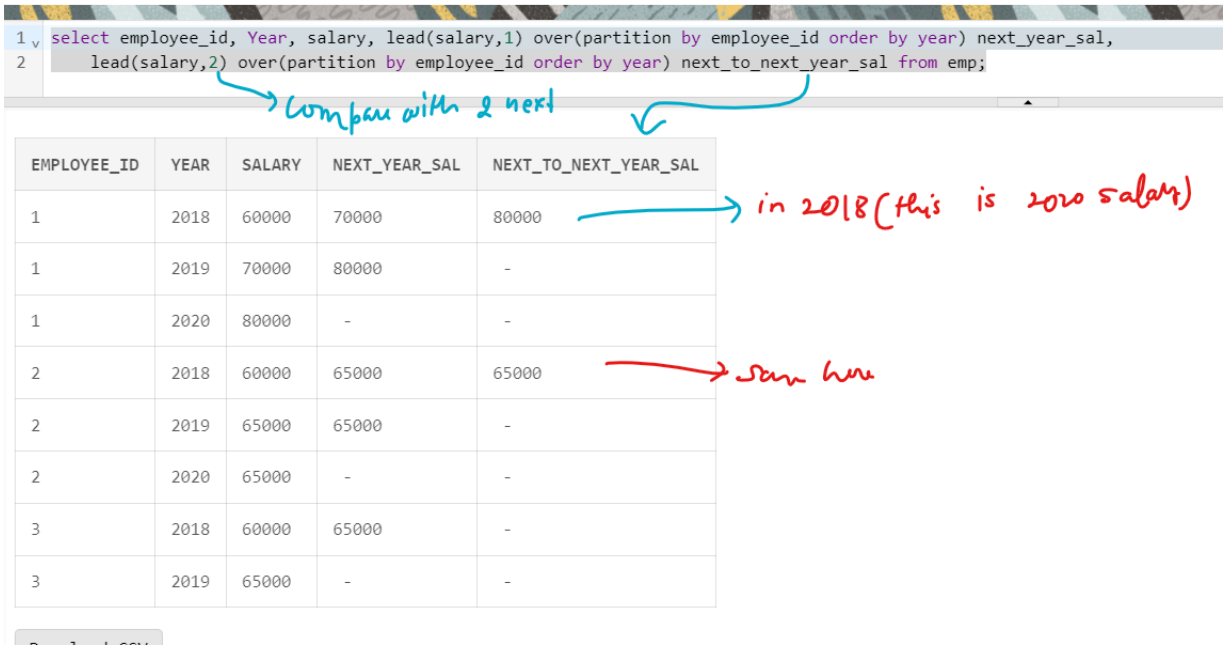


EMPLOYEE_ID	SALARY	YEAR	NEXT_YEAR_SAL
1	60000	2018	70000
1	70000	2019	80000
1	80000	2020	-
2	60000	2018	65000
2	65000	2019	65000
2	65000	2020	-
3	60000	2018	65000
3	65000	2019	-

See the above sql, how it work, it simple print next_year_sal by using lead function. for 2018, next year is 2019, and for 2019 the next year is 2020.
In the same, because we want to compare 3 years salary, we want 1 more year to compare

lead(salary,2) over(partition by employee_id order by year) next_to_next_year_sal

lead --> next year
salary --> this is what i want to compare
2 --> 2 next years



EMPLOYEE_ID	YEAR	SALARY	NEXT_YEAR_SAL	NEXT_TO_NEXT_YEAR_SAL
1	2018	60000	70000	80000
1	2019	70000	80000	-
1	2020	80000	-	-
2	2018	60000	65000	65000
2	2019	65000	65000	-
2	2020	65000	-	-
3	2018	60000	65000	-
3	2019	65000	-	-

Now I have all the data for 3 years to compare. I will simply write where **next_year_sal > salary and next_to_next_year_sal > next_year_sal**

```
with n1 as (select employee_id, Year, salary, lead(salary,1) over(partition by employee_id order by year) next_year_sal,
   lead(salary,2) over(partition by employee_id order by year) next_to_next_year_sal from emp)
select * from N1 where next_year_sal > salary and next_to_next_year_sal > next_year_sal
```

```
with n1 as (select employee_id, Year, salary, lead(salary,1) over(partition by employee_id order by year) next_year_sal,
    lead(salary,2) over(partition by employee_id order by year) next_to_next_year_sal from emp)
select * from N1 where next_year_sal > salary and next_to_next_year_sal > next_year_sal
```

EMPLOYEE_ID	YEAR	SALARY	NEXT_YEAR_SAL	NEXT_TO_NEXT_YEAR_SAL
1	2018	60000	70000	80000

got the data

==========

```
create table test (year number,brand varchar2(30) ,amount number ) ;

insert into test values(2018,'apple',45000);
insert into test values(2019,'apple',35000);
insert into test values(2020,'apple',75000);
insert into test values(2018,'samsung',15000);
insert into test values(2019,'samsung',20000);
insert into test values(2020,'samsung',25000);
insert into test values(2018,'nokia',21000);
insert into test values(2019,'nokia',17000);
insert into test values(2020,'nokia',14000);
```

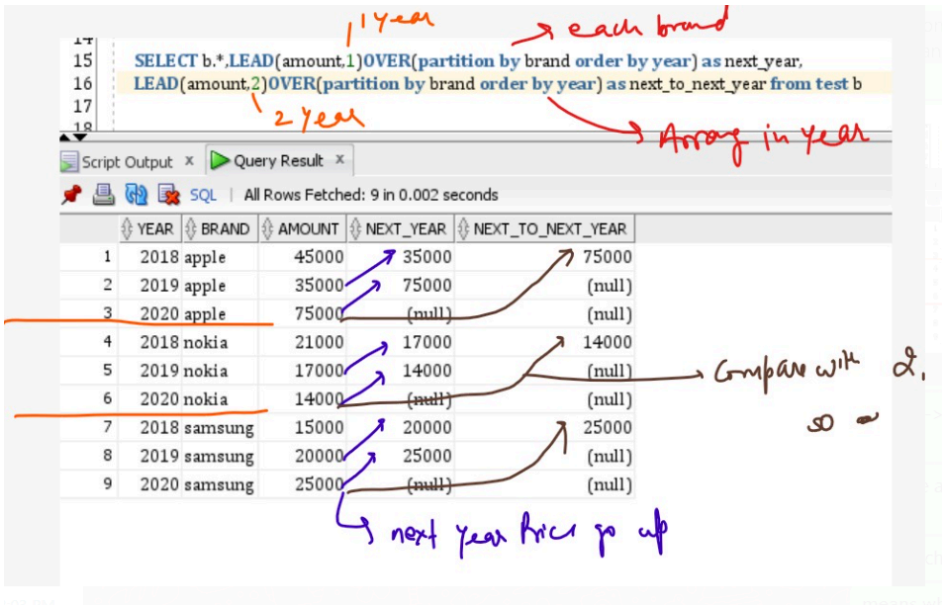
```
12  select * from test;
```

Script Output | Query... | SQL | All Rows Fetched: 9 in 0.002 seconds

	YEAR	BRAND	AMOUNT
1	2018	apple	45000
2	2019	apple	35000
3	2020	apple	75000
4	2018	samasung	15000
5	2019	samsung	20000
6	2020	samsung	25000
7	2018	nokia	21000
8	2019	nokia	17000
9	2020	nokia	14000

write the query to featch the record of brand whose amount is increasing every year ?
Understand the question every means --> group by.. but we want to comapre price each year, right
compare means : lead and lag
lead and lag means analytical
and analytical means --> group by chnage to partition by

```
SELECT b.*,LEAD(amount,1)OVER(partition by brand order by year) as next_year,
   LEAD(amount,2)OVER(partition by brand order by year) as next_to_next_year from test b
```



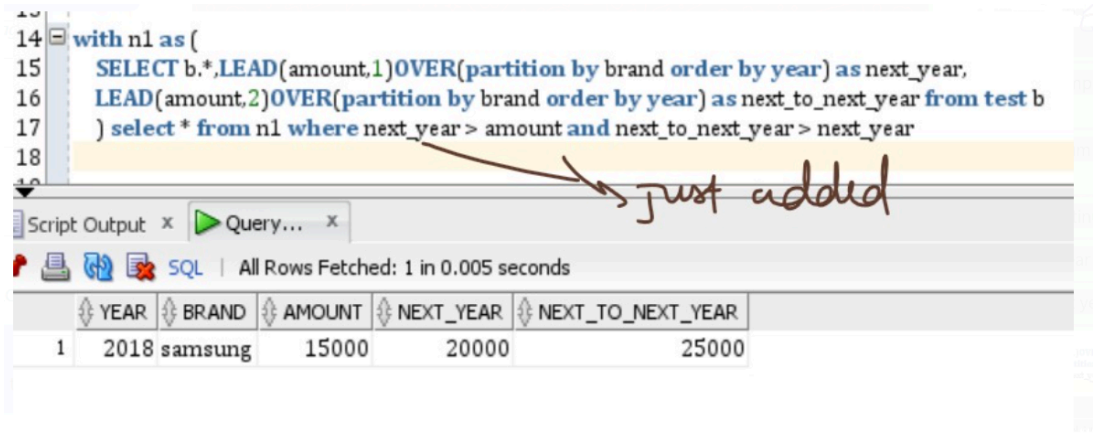

```
SELECT b.*,LEAD(amount,1)OVER(partition by brand order by year) as next_year,
LEAD(amount,2)OVER(partition by brand order by year) as next_to_next_year from test b
```

Script Output | Query Result

SQL | All Rows Fetched: 9 in 0.002 seconds

	YEAR	BRAND	AMOUNT	NEXT_YEAR	NEXT_TO_NEXT_YEAR
1	2018	apple	45000	35000	75000
2	2019	apple	35000	75000	(null)
3	2020	apple	75000	(null)	(null)
4	2018	nokia	21000	17000	14000
5	2019	nokia	17000	14000	(null)
6	2020	nokia	14000	(null)	(null)
7	2018	samsung	15000	20000	25000
8	2019	samsung	20000	25000	(null)
9	2020	samsung	25000	(null)	(null)

Now see here, i wrote lead (amount,1) --> means compare the amount column with 1 means 1 next
lead (amount,2) --> means compare the amount column with 2 means 2 next
Now what is the question, which brand continuously increase price
means where next year > amount and next to next year > next year price




```
with n1 as (
  SELECT b.*,LEAD(amount,1)OVER(partition by brand order by year) as next_year,
  LEAD(amount,2)OVER(partition by brand order by year) as next_to_next_year from test b
  ) select * from n1 where next_year > amount and next_to_next_year > next_year
```

Script Output | Query...

SQL | All Rows Fetched: 1 in 0.005 seconds

	YEAR	BRAND	AMOUNT	NEXT_YEAR	NEXT_TO_NEXT_YEAR
1	2018	samsung	15000	20000	25000